

Seminario de Actualización

me sugirio instalar esto VS:

no se actualiza la pantalla

Excelente. Para el Seminario de Actualización, yo haría una versión mucho más desarrollada, prácticamente como un capítulo de apuntes. La estructura podría ser la siguiente:

Seminario de Actualización

Unidad Práctica Integrada

Comparación entre MVC Tradicional y Frontend Desacoplado

1. Introducción

Durante muchos años el desarrollo web estuvo basado en arquitecturas donde el servidor generaba directamente las páginas HTML consumidas por el navegador.

Con la evolución de Internet y el crecimiento de las aplicaciones web surgieron nuevas arquitecturas que separan completamente frontend y backend.

El objetivo de esta actividad es comparar:

- MVC tradicional.
- Frontend desacoplado.
- APIs REST.
- React.
- Servlets.
- Docker.

Analizando ventajas, desventajas y escenarios de uso.

2. Objetivos

Al finalizar la actividad el estudiante será capaz de:

- Comprender el funcionamiento del patrón MVC.
- Entender el concepto de frontend desacoplado.
- Consumir APIs REST.
- Identificar el rol de React.
- Comprender el funcionamiento de Servlets.
- Utilizar Docker para desplegar aplicaciones.

3. Arquitectura MVC Tradicional

¿Qué es MVC?

MVC significa:

Modelo
Vista
Controlador

Es un patrón de diseño que divide una aplicación en tres componentes.

Modelo

Representa:

- datos,
- lógica de negocio,
- acceso a información.

Ejemplo:

Java

Alumno
Cuenta
Cliente
Vehiculo

Vista

Representa la interfaz.

En Java tradicional:

```
JSP
HTML
CSS
```

Controlador

Recibe las peticiones del usuario.

En Java Web:

```
</> Java
Servlet
```

Diagrama MVC

```
Usuario
  ↓
Vista (JSP)
  ↓
Controlador (Servlet)
  ↓
Modelo
  ↑
Vista
```

Flujo MVC

Paso 1

Usuario solicita:

```
/alumnos
```

Paso 2

Tomcat recibe la petición.

Paso 3

El Servlet Controller procesa la solicitud.

Paso 4

Obtiene datos del modelo.

Paso 5

Coloca los datos en:

```
</> Java  
request.setAttribute(...)
```

Paso 6

Realiza:

```
</> Java  
forward(...)
```

hacia un JSP.

Paso 7

El JSP genera HTML.

Paso 8

El navegador muestra la página.

Arquitectura MVC

```
Browser  
↓  
Servlet Controller  
↓
```

Modelo



JSP



HTML

4. Implementación MVC desarrollada

Modelo

Java

```
public class Alumno {  
  
    private int id;  
    private String nombre;  
  
}
```

Controlador

Java

```
@WebServlet("/alumnos")  
public class AlumnoController
```

Vista

alumnos.jsp

Operaciones realizadas

GET

Obtiene alumnos.

Java

```
doGet()
```

POST

Agrega alumnos.

```
</> Java  
doPost()
```

Limitación

Los alumnos se almacenan en memoria:

```
</> Java  
List<Alumno>
```

Por lo tanto:

```
Si se reinicia el contenedor  
se pierde la información
```

5. Limitaciones del MVC tradicional

MVC continúa siendo válido, pero presenta algunas limitaciones:

- frontend y backend acoplados,
- difícil reutilización,
- complejidad para aplicaciones móviles,
- menor flexibilidad tecnológica.

6. Evolución hacia APIs REST

Con el crecimiento de:

- aplicaciones móviles,
- sistemas distribuidos,
- cloud computing,

surgió la necesidad de desacoplar frontend y backend.

API

API significa:

Application Programming Interface

Permite que distintas aplicaciones intercambien información.

REST

REST es un estilo arquitectónico utilizado para construir servicios web.

Métodos HTTP

GET

Obtiene información.

POST

Crea información.

PUT

Actualiza información.

DELETE

Elimina información.

Ejemplo

```
</> http  
GET /api/alumnos
```

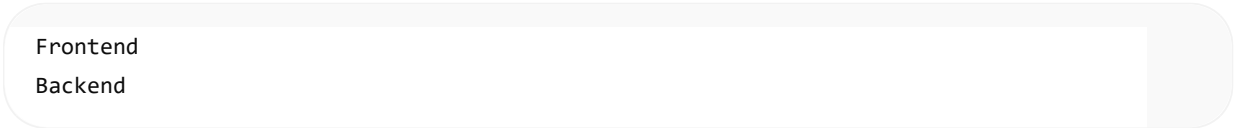
Respuesta

```
</> JSON
```

```
[  
  {  
    "id": 1,  
    "nombre": "Ana"  
  }  
]
```

7. Frontend desacoplado

En una arquitectura moderna:



Frontend
Backend

son aplicaciones independientes.

Frontend

Puede desarrollarse utilizando:

- React
- Vue
- Angular

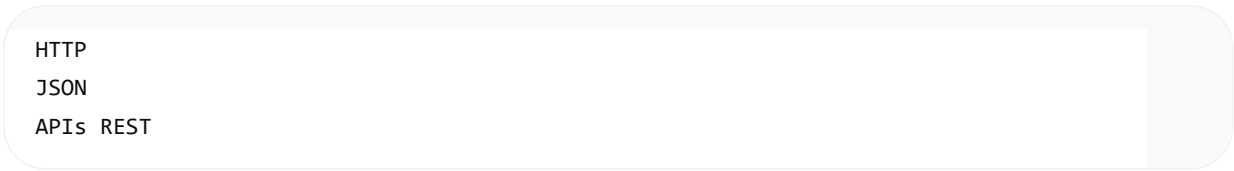
Backend

Puede desarrollarse utilizando:

- Java
- Spring Boot
- Node.js
- Django

Comunicación

Se realiza mediante:



HTTP
JSON
APIs REST

Arquitectura moderna

```
React
  ↓
fetch()
  ↓
API REST
  ↓
Servlet
```

8. React

React es una librería JavaScript utilizada para construir interfaces modernas.

Ventajas

- componentes reutilizables,
- interfaces dinámicas,
- SPA,
- gran ecosistema.

Rol de React en el proyecto

React:

- renderiza la interfaz,
- consume APIs REST,
- actualiza la pantalla dinámicamente.

Archivo principal

```
App.jsx
```

Consumo de API

```
</> JavaScript
```

```
fetch(  
  "http://localhost:8080/backend-servlet/api/alumnos"  
)
```

¿Qué hace fetch?

Genera una petición HTTP.

Flujo completo

```
React  
↓  
fetch()  
↓  
GET  
↓  
Servlet  
↓  
JSON  
↓  
React
```

9. Vite

Vite es una herramienta de desarrollo frontend.

Funciones

- crea proyectos React,
- servidor de desarrollo,
- build,
- recarga automática.

Comando utilizado

```
</> Bash  
npm create vite@latest frontend-react
```

10. Node.js

Node.js permite ejecutar JavaScript fuera del navegador.

Utilización

En este proyecto:

```
npm  
Vite  
React Tooling
```

Node NO ejecuta React

React finalmente se ejecuta en:

```
el navegador
```

11. Docker

Docker permite ejecutar aplicaciones dentro de contenedores.

Problema que resuelve

```
En mi computadora funciona
```

Imagen

Plantilla preparada.

Contenedor

Instancia ejecutándose.

Dockerfile utilizado

</> dockerfile

FROM tomcat:10.1-jdk17

COPY target/backend-servlet.war /usr/local/tomcat/webapps/

EXPOSE 8080

Flujo Docker

Código Java



WAR



Imagen Docker



Contenedor



Tomcat



Aplicación funcionando

12. Comparación MVC vs Frontend Desacoplado

MVC Tradicional	Frontend Desacoplado
JSP	React
HTML generado en servidor	HTML generado en navegador
Servlet → JSP	Servlet → JSON
Acoplado	Desacoplado
Menor reutilización	Alta reutilización
Aplicaciones web	Web + Mobile + APIs

13. Actividades prácticas propuestas

Actividad 1

Agregar alumnos desde formulario MVC.

Actividad 2

Agregar alumnos mediante React.

Actividad 3

Implementar:

GET
POST

Actividad 4

Agregar:

PUT
DELETE

Actividad 5

Persistir información utilizando:

JDBC + MySQL

Actividad 6

Ejecutar:

Tomcat Docker
MySQL Docker

mediante Docker Compose.

14. Preguntas para discusión

1. ¿Qué ventajas presenta React frente a JSP?
2. ¿Por qué surgieron las APIs REST?
3. ¿Qué beneficios aporta Docker?
4. ¿Qué ocurre con los datos cuando se reinicia el contenedor?
5. ¿Por qué el frontend desacoplado facilita el desarrollo móvil?
6. ¿Qué ventajas presenta una arquitectura basada en microservicios?

15. Conclusiones

La actividad permitió comprender la evolución de las arquitecturas web desde el modelo MVC tradicional hasta las arquitecturas modernas basadas en frontend desacoplado.

Además se integraron tecnologías ampliamente utilizadas en el mercado profesional:

- Java,
- Servlets,
- React,
- APIs REST,
- Docker,
- JSON,
- HTTP.

Estos conceptos constituyen la base de gran parte de las aplicaciones empresariales modernas y representan una introducción práctica a arquitecturas distribuidas, cloud y microservicios.



todo esto mismo que desarrollaste en un PDF incluyendo el paso a paso de las actividades

Preparé el PDF:



Observación: este PDF contiene una versión resumida estructurada de los temas y actividades. Si querés un **apunte de cátedra completo (15-20 páginas)** con:

- desarrollo teórico detallado,
- diagramas ASCII,

- código completo de los proyectos,
- paso a paso completo de React + Docker + Servlet,
- paso a paso completo de MVC + JSP,
- capturas/explicaciones de `fetch()`, `doGet()`, `doPost()`,
- Dockerfile comentado,
- ejercicios y actividades de evaluación,

puedo generarte una segunda versión mucho más extensa y académica, pensada directamente para entregar a los alumnos.



dame esa segunda version mucho más extensa y académica para los alumnos

Excelente. Para alumnos de 3° año de Analista de Sistemas, yo la transformaría en una **Unidad Práctica Integrada** de aproximadamente 15 a 20 páginas, con un enfoque académico, pero manteniendo el perfil profesional que busca el Seminario de Actualización.

SEMINARIO DE ACTUALIZACIÓN

UNIDAD PRÁCTICA INTEGRADA

Evolución de las Arquitecturas Web: MVC Tradicional y Frontend Desacoplado

1. Introducción

Durante las últimas décadas el desarrollo de aplicaciones web ha evolucionado significativamente.

Las primeras aplicaciones web consistían principalmente en páginas estáticas que eran enviadas por un servidor hacia un navegador.

Con el crecimiento de Internet y el aumento de la complejidad de los sistemas informáticos surgieron nuevas arquitecturas destinadas a mejorar:

- la organización del software,
- la reutilización de componentes,
- la escalabilidad,

- el mantenimiento,
- la experiencia del usuario.

Dentro de esta evolución aparece el patrón MVC (Modelo-Vista-Controlador), ampliamente utilizado durante muchos años en aplicaciones empresariales.

Posteriormente, el crecimiento de las aplicaciones móviles, los sistemas distribuidos y los servicios cloud impulsó el desarrollo de arquitecturas desacopladas basadas en APIs REST y frameworks frontend modernos como React, Vue y Angular.

La presente actividad tiene como objetivo analizar y comparar ambos enfoques mediante la implementación de ejemplos funcionales.

2. Objetivos de Aprendizaje

Al finalizar esta actividad el estudiante será capaz de:

Conceptuales

- Comprender el patrón MVC.
- Entender el funcionamiento de una API REST.
- Diferenciar frontend y backend.
- Analizar la evolución de las arquitecturas web.
- Comprender el rol de Docker en el despliegue moderno.

Procedimentales

- Implementar un Servlet Java.
- Crear una vista JSP.
- Construir una API REST.
- Consumir una API desde React.
- Generar imágenes Docker.
- Ejecutar aplicaciones en contenedores.

Actitudinales

- Analizar críticamente tecnologías modernas.
 - Comprender la importancia de la evolución tecnológica.
 - Valorar la necesidad del aprendizaje continuo.
-

3. Arquitectura Cliente-Servidor

Toda aplicación web moderna se basa en el modelo cliente-servidor.

Cliente

El cliente es la aplicación que solicita información.

Ejemplos:

- Navegador web.
 - Aplicación móvil.
 - React.
 - Angular.
 - Vue.
-

Servidor

El servidor recibe solicitudes y responde información.

Ejemplos:

- Tomcat.
 - Spring Boot.
 - Node.js.
 - Django.
-

Comunicación

La comunicación generalmente se realiza mediante:

HTTP
HTTPS
JSON
XML

Diagrama general

Cliente
↓ HTTP
Servidor
↓
Base de datos

4. Patrón MVC

MVC significa:

```
Modelo  
Vista  
Controlador
```

Es uno de los patrones arquitectónicos más importantes en la historia del desarrollo web.

Objetivos del patrón

MVC busca:

- separar responsabilidades,
 - facilitar mantenimiento,
 - mejorar reutilización,
 - simplificar el trabajo colaborativo.
-

Modelo

Representa:

- datos,
- lógica de negocio,
- reglas del sistema.

Ejemplos:

```
</> Java
```

```
Alumno  
Cliente  
Producto  
Cuenta
```

Vista

Representa la interfaz de usuario.

En Java tradicional:

```
JSP  
HTML
```

CSS

Controlador

Coordina el funcionamiento del sistema.

En aplicaciones Java Web:

`</> Java``Servlet`

Flujo MVC

```
Usuario
  ↓
Vista
  ↓
Controlador
  ↓
Modelo
  ↑
Vista
```

5. Implementación MVC Tradicional

Escenario

Sistema simple de alumnos.

Modelo

`</> Java``Alumno`

Contiene:

- id
- nombre

Controlador

Java

AlumnoController

Responsabilidades:

- recibir peticiones,
- obtener información,
- reenviar a JSP.

Vista

alumnos.jsp

Responsabilidades:

- mostrar información,
- renderizar HTML.

Flujo real

```
Browser
↓
GET /alumnos
↓
AlumnoController
↓
List<Alumno>
↓
request.setAttribute()
↓
forward()
↓
alumnos.jsp
↓
HTML
```

6. Limitaciones del MVC Tradicional

Aunque MVC continúa utilizándose, presenta algunas limitaciones.

Acoplamiento

Frontend y backend se encuentran fuertemente vinculados.

Reutilización limitada

La información generada está pensada para navegadores.

Aplicaciones móviles

Dificulta reutilizar lógica para:

- Android,
 - iOS,
 - aplicaciones externas.
-

Escalabilidad tecnológica

Resulta más difícil reemplazar únicamente el frontend.

7. Evolución hacia APIs REST

Con la expansión de Internet surgió la necesidad de compartir información entre múltiples aplicaciones.

¿Qué es una API?

API significa:

Application Programming Interface

Una API permite que diferentes aplicaciones intercambien información.

REST

REST significa:

Representational State Transfer

Es un estilo arquitectónico ampliamente utilizado para construir servicios web.

Principales métodos HTTP

GET

Obtiene información.

POST

Crea información.

PUT

Actualiza información.

DELETE

Elimina información.

Ejemplo

```
</> http
GET /api/alumnos
```

Respuesta:

```
</> JSON
[
  {
    "id": 1,
    "nombre": "Ana"
  }
]
```

8. JSON

JSON significa:

JavaScript Object Notation

Es el formato más utilizado actualmente para intercambiar información.

Ventajas

- legible,
 - liviano,
 - multiplataforma,
 - ampliamente soportado.
-

Ejemplo

JSON

```
{  
  "id": 1,  
  "nombre": "Ana"  
}
```

9. Frontend Desacoplado

En una arquitectura moderna:

Frontend
Backend

son aplicaciones independientes.

Frontend

Puede desarrollarse utilizando:

- React,
 - Vue,
 - Angular.
-

Backend

Puede desarrollarse utilizando:

- Java,
- Spring,
- Node.js,
- Python.

Comunicación

HTTP
JSON
REST

Arquitectura

React
↓
API REST
↓
Servlet

10. React

React es una librería JavaScript creada para construir interfaces modernas.

Características

- Componentes.
- Estado.
- Renderizado dinámico.
- SPA.

SPA

SPA significa:

Single Page Application

La página no se recarga completamente.

Ejemplo utilizado

JavaScript

```
fetch("http://localhost:8080/backend-servlet/api/alumnos")
```

¿Qué ocurre?

```
graph TD; React1[React] --> HTTP[HTTP GET]; HTTP --> Servlet[Servlet]; Servlet --> JSON[JSON]; JSON --> React2[React];
```

React
↓
HTTP GET
↓
Servlet
↓
JSON
↓
React

11. Vite y Node.js

Node.js

Permite ejecutar JavaScript fuera del navegador.

npm

Administrador de paquetes.

Vite

Herramienta moderna para:

- crear proyectos,
- compilar,
- ejecutar React.

Comandos utilizados

</> Bash

```
npm create vite@latest frontend-react
npm install
npm run dev
```

12. Docker

Docker es una plataforma de contenedores.

Problema que resuelve

En mi computadora funciona

Imagen

Plantilla preparada.

Contenedor

Instancia ejecutándose.

Dockerfile utilizado

</> dockerfile

```
FROM tomcat:10.1-jdk17

COPY target/backend-servlet.war /usr/local/tomcat/webapps/

EXPOSE 8080
```

Flujo Docker

Código Java



WAR



Imagen



Contenedor



Tomcat

13. Actividad Práctica 1

MVC Tradicional

Objetivo

Implementar:

Servlet

+

JSP

+

Formulario HTML

Paso 1

Crear proyecto Maven WAR.

Paso 2

Crear modelo Alumno.

Paso 3

Crear AlumnoController.

Paso 4

Crear JSP.

Paso 5

Compilar.

</> Bash

```
mvn clean package
```

Paso 6

Dockerizar.

</> Bash

```
docker build -t backend-servlet-mvc .
```

Paso 7

Ejecutar.

</> Bash

```
docker run -p 8081:8080 backend-servlet-mvc
```

14. Actividad Práctica 2

Frontend Desacoplado

Objetivo

Implementar:

```
React
+
API REST
+
Docker
```

Paso 1

Crear proyecto backend.

Paso 2

Crear Servlet REST.

```
</> Java
@WebServlet("/api/alumnos")
```

Paso 3

Compilar WAR.

Paso 4

Dockerizar Tomcat.

Paso 5

Crear React.

```
</> Bash
npm create vite@latest frontend-react
```

Paso 6

Consumir API.

```
</> JavaScript
fetch(...)
```

Paso 7

Implementar POST.

Paso 8

Actualizar listado dinámicamente.

15. Comparación Final

MVC Tradicional	Frontend Desacoplado
JSP	React
HTML servidor	HTML navegador
Acoplado	Desacoplado
Servlet → JSP	Servlet → JSON
Web tradicional	Aplicaciones modernas

16. Preguntas de Reflexión

1. ¿Qué ventajas aporta React respecto de JSP?
2. ¿Por qué surgieron las APIs REST?
3. ¿Qué beneficios aporta Docker?
4. ¿Qué ocurre con los datos al reiniciar un contenedor?
5. ¿Por qué una API puede ser utilizada por múltiples clientes?
6. ¿Cómo se relacionan estas arquitecturas con microservicios?

17. Conclusiones

La evolución desde MVC tradicional hacia arquitecturas desacopladas refleja la transformación de la industria del software.

Actualmente gran parte de las aplicaciones empresariales utilizan:

- APIs REST,
- Frontend desacoplado,
- React,
- Docker,
- Cloud Computing.

Comprender ambas arquitecturas permite al futuro Analista de Sistemas interpretar tanto sistemas heredados como soluciones modernas, facilitando su adaptación a los cambios tecnológicos del mercado profesional.



